



DIPARTIMENTO DI INGEGNERIA INFORMATICA, MODELLISTICA,
ELETTRONICA E SISTEMISTICA

Robotics and Automation Engineering

Embedded Systems

Ping pong ball levitation with hand control

Professori:

Prof. Gianni Cario

Prof. Franco Angelo

Torchiaro

Studentessa:

Francesca Vaccaro

Mat. 276582

ANNO ACCADEMICO 25/26

Indice

Indice	1
1 Introduction and hardware setting	3
1.1 Main object of the project	3
1.2 Hardware composition	4
1.2.1 Ultrasonic Proximity sensor	6
1.3 Logic part	8
1.3.1 F446RE	9
GPIOs	9
Timers	10
Interrupts	14
1.3.2 Sensor Reading Implementation	17
1.3.3 Driver tb6612fng	18
1.3.4 Buck converter lm2596	19
2 Serial Communication and Filter	20
2.1 Serial Communication (USART)	20
2.2 Data Filtering (EMA)	23
3 PID controller	24
3.1 Control Strategy and PID Implementation	24
3.1.1 Proportional, derivative and integral gain	26
Proportional gain	26
Derivative gain	26
Integral action	26
4 Graphics evaluation	27

5 Conclusion and Future Developments	29
5.1 Project Conclusions	29
5.2 Future Developments and Industrial Applications	30
Bibliografia	31

Capitolo 1

Introduction and hardware setting

1.1 Main object of the project

The aim of this project consists of designing an embedded system capable to maintains a **levitating ping pong ball at a dynamic target height**, dictated in real-time by the **position of a user's hand**. In this chapter is possible to observe the structure both from hardware and logic/software view.

1.2 Hardware composition



Figura 1.1: Immagine frontale del progetto

As shown in the first frontal photo 1.1, the physical setup is built around a transparent *plexiglass tube* that houses the levitating object. Plexiglass was selected for its structural rigidity and transparency, allowing full observation of the system's dynamics. The inner diameter was specifically chosen to accommodate a standard 40 mm ping-pong ball, providing sufficient clearance for vertical movement while tightly restricting lateral bouncing and turbulence. A standard *ping-pong ball* was selected due to its optimal weight-to-surface-area ratio. While a lighter polystyrene sphere was considered as an alternative, its porous and irregular surface would scatter ultrasonic waves, leading to heavily distorted sensor readings. The hard plastic shell of the ping-pong ball ensures a crisp acoustic reflection, which is strictly required for accurate feedback.

Finally, the lower section of the image displays the system's actuator: a 5V *DC centrifugal blower fan (5015 model)*. Unlike standard axial fans, this radial configuration generates the high static pressure required to force air through the cylindrical chamber and overcome the sphere's gravitational pull.



Figura 1.2: Immagine laterale del progetto



Figura 1.3: Immagine superiore del progetto

Moving to the side and top views 1.2 1.3, the images illustrate the deployment of the *two HC-SR04 ultrasonic sensors*:

1. **Feedback Sensor**: Mounted above the tube (with sufficient clearance to avoid obstructing the upward exhaust airflow), this sensor measures the absolute distance of the ball from the upper zero-reference point.
2. **Setpoint Sensor**: Mounted at the base and angled parallel to the tube's axis, this sensor tracks the distance of the user's hand to dynamically generate the target height.

1.2.1 Ultrasonic Proximity sensor

Sensors of this type generally use an **ultrasonic signal**, with a frequency of *40 KHz*, to determine the distance from what they have in front of them. The

transmitted wave is reflected by the obstacle and goes back to the sensor which determines the time of flight of the wave and, knowing the speed of sound in the air, allows the calculation of the distance from the obstacle.

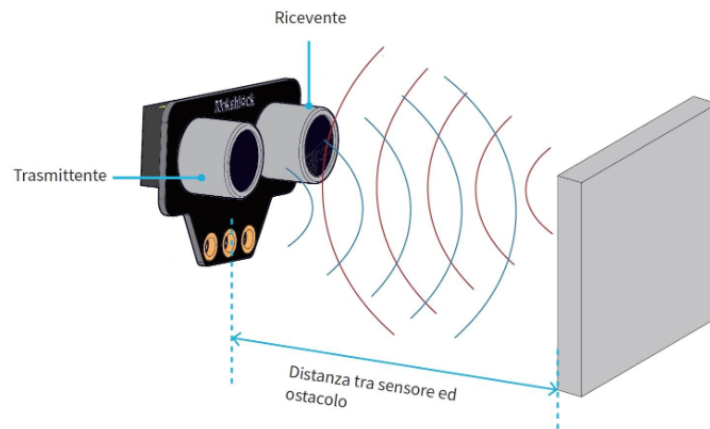


Figura 1.4: Sensor

The module used in this project is **HC-SR04**, which measures distances between 2 cm and 4 m with a resolution of up to 3 mm. It is a module that contains both transmitter and receiver, located on the same axis.

To acquire a valid measurement, the system executes the following operational sequence:

1. keep the trig high for $10\mu s$;
2. Wait for the 8 ultrasound cycles at 40 KHz without doing anything;
3. When the signal starts to go back, intercept the high value of the ECHO pin and calculate the time it remains high;
4. Determine the distance using the duration (in μs) of the high ECHO signal and the speed of sound in air;
5. Wait at least 60 ms before taking a new measurement.

1.3 Logic part

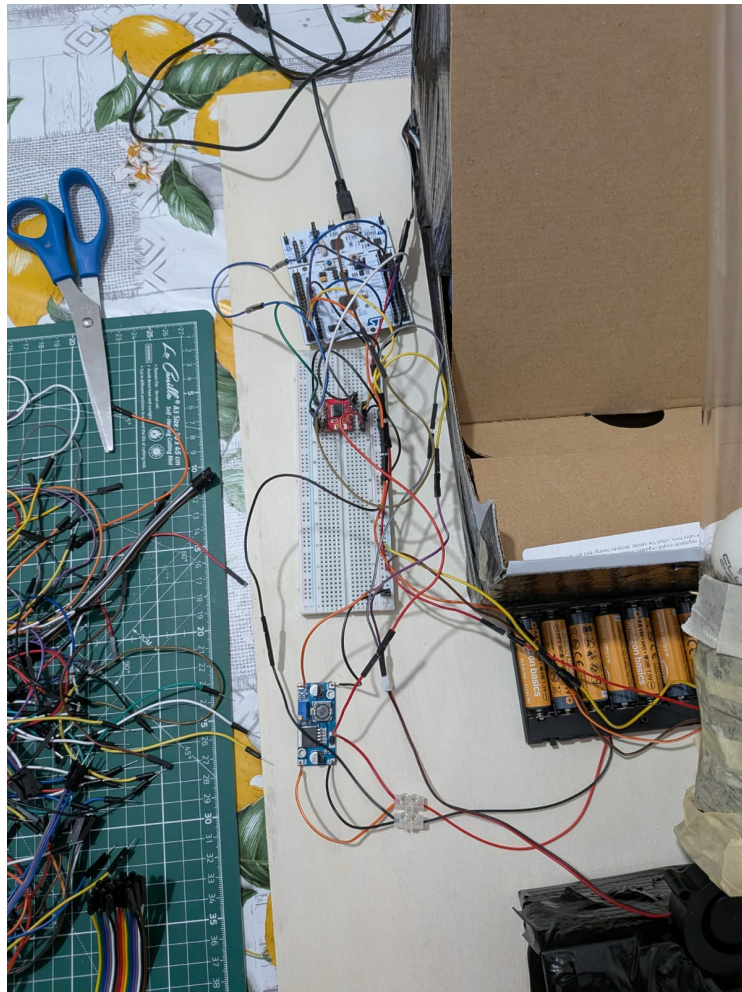


Figura 1.5: Circuito

In this context, we can observe the project's electric and logic motor. This is where the sensors and actuator will communicate, ensuring everything is as smooth as possible and safe as possible in terms of current output.

Let's start with the main power source, the **battery compartment**, which houses eight AA batteries. This choice was made to provide 12V, which allows the fan to operate at maximum power. However, 12V cannot be supplied to all components as it would create short circuits. In fact, the board and sensors require a maximum of 5V, so a **buck converter** was installed to convert the 12V to 5V. This was achieved using mammutts to safely separate the two cables. And then we got **the F446RE** board is the core of the project.

So let's see in details those elements.

1.3.1 F446RE

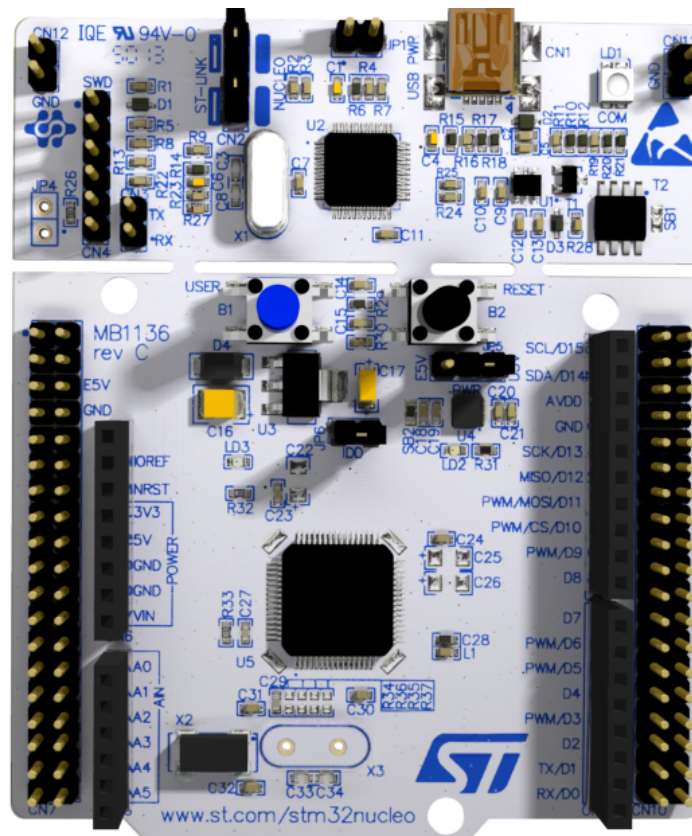


Figura 1.6: microcontroller

Starting from the STM32 Nucleo-64 board, this is a **microcontroller** that interfaces with sensors, actuators and communication module. with this is possible to manage **GPIOs**, **interrupt**, **timers** and a lot of other stuff. The core architecture is based on a 32bit ARM Cortex-M4 with an hardware FPU(Floating-Point Unit).

GPIOs

As previously introduced, the board communicates with external peripherals through GPIO (General Purpose Input/Output) interfaces. Working with a bare-metal approach, the proper initialization of these interfaces requires a rigorous hardware design process divided into three phases: analyzing the Pin Map, studying the Reference Manual, and implementing the software through direct register manipulation.

So the steps are :

1. Consult the STM32 Nucleo-F446RE datasheet to map logical signals to physical pins.
2. For the hardware PWM signal required to drive the motor, the microcontroller's Pin Map indicates that Timer 3 Channel 1 (*TIM3_CH1*) can be routed to pin PA6 (Port A, pin 6).
3. Once the pin is identified, the microcontroller's Reference Manual is consulted to determine the exact binary combinations required.
4. To delegate control of the PA6 pin to the hardware PWM generator, the MODER register must switch from the standard Output mode to "Alternate Function" mode (binary value 10, or decimal 2).
5. The manual also specifies configuring the internal multiplexer via the AFR (Alternate Function Register). The alternate function table links TIM3 to the AF2 label (hexadecimal value 0x02).

This theoretical process is finally translated into code within the `TIM3_PWM_setup()` function. Using bitmasks and logical operations (`=` for clearing, `—=` for setting), the registers are configured atomically. This modifies only the specific bits dedicated to PA6 (bits 12 and 13) without corrupting the state of the other pins on the same port:

```

1
2 RCC->AHB1ENR |= (1 << RCC_AHB1ENR_GPIOAEN_Pos);
3 GPIOA->MODER &= ~(3 << 12);
4 GPIOA->MODER |= (2 << 12);
5 GPIOA->AFR[0] &= ~(0xF << 24);
6 GPIOA->AFR[0] |= (2 << 24);
7 GPIOA->OSPEEDR |= (3 << 12);

```

Listing 1.1: implementation

Timers

Many times in the development of a program it is necessary, for various reasons, to wait for something to happen. It can be possible to use `HAL_Delay()` in a

very simple and straightforward way but such a function is blocking function for the microcontroller.

In most embedded systems, the exact knowledge of how much time has passed can be essential for time management and often providing an immediate response to a certain event is crucial. A wait that makes the CPU "waste time" is not appropriate and that's why all microcontrollers provide dedicated hardware peripherals for time management: Timers.

A timer, as a hardware device, is a counter that performs its count operations with a frequency that is somehow related to the clock source of the MCU, both internal or external. This count is carried out by increasing (or decreasing) the value contained in suitable registers until an overflow situation occurs, which causes an interrupt. The maximum value beyond which the overflow occurs depends on the timer resolution (for example, for a timer with 16 bit resolution the maximum value is 65535).

According to its type, a timer can be used in different modes, the main ones are: compare mode, capture mode, and PWM mode. These allow the programmer to use a timer not only to perform counter functions but also to create more complex functions. In the next part of the lesson we will analyze, as already specified, what are the main ingredients (the internal components and main registers) in order to set up these modes. Note that a general purpose timer can be used to serve the same purposes as a basic timer, but the contrary is not true.

The timing requirements of the aerodynamic levitation system are strictly managed by two independent hardware timers, configured via direct register manipulation to prevent CPU blocking delays.

```
1
2 void TIM6_DAC_IRQHandler() {
3     if((TIM6->SR >> TIM_SR_UIF_Pos) & 0x01) {
4         NVIC_ClearPendingIRQ(TIM6_DAC_IRQn); // Eliminare richieste
5         // di interrupt pendenti
6         time_elapsed = 1;
7         TIM6->CR1 &= ~(1 << TIM_CR1_CEN_Pos);
8         TIM6->SR &= ~(1 << TIM_SR_UIF_Pos);
9     }
```

```

9 }
10
11 void TIM6_basic_setup() {
12
13     RCC->APB1ENR |= (1 << RCC_APB1ENR_TIM6EN_Pos);    // Clock to
        TIM6
14
15     TIM6->CR1 &= ~(1 << TIM_CR1_UDIS_Pos);    // Update event
        enable
16     TIM6->CR1 |= (1 << TIM_CR1_URS_Pos);    // Update event
        generated only by overflow/underflow
17     //TIM6->PSC = 90 - 1;    // CNT incremented every
        nanoseconds perch il codice lavora solo se il clock
        90Mhz quindi 90/90 = 1hz -> frequenza che ci serve per i //
        microsecondi
18     TIM6->PSC = 84 - 1;
19
20     TIM6->DIER |= (1 << TIM_DIER_UIE_Pos);    // Trigger an
        interrupt every update event
21     NVIC_EnableIRQ(TIM6_DAC_IRQn);    // Enable interrupt for
        TIM6
22     NVIC_SetPriority(TIM6_DAC_IRQn, 0);    // Set Priority
23 }
24
25
26 void TIM3_PWM_setup() {
27     /* GPIO SetUp (PA6 -> TIM3_CH1) */
28     RCC->AHB1ENR |= (1 << RCC_AHB1ENR_GPIOAEN_Pos);    // Enable
        GPIO6
29     /*GPIOA->MODER |= (2 << 12);    // Alternate
        function mode
30     GPIOA->AFR[0] = (2 << 24);
31     /*    // Alternate function n2 for PA6 (TIM3 CH1)
32     GPIOA->MODER &= ~(3 << 12);
33     GPIOA->MODER |= (2 << 12);
34
35     GPIOA->AFR[0] &= ~(0xF << 24);
36     GPIOA->AFR[0] |= (2 << 24);
37     GPIOA->OSPEEDR |= (3 << 12);    // High Speed for
        PIN PA6

```

```

38
39  /* TIM3 SetUp */
40  RCC->APB1ENR |= (1 << RCC_APB1ENR_TIM3EN_Pos);    // Clock to
    TIM3
41
42  /* CR1 Configuration */
43  TIM3->CR1 &= ~(1 << TIM_CR1_UDIS_Pos);    // Update event
    enabled
44  TIM3->CR1 &= ~(1 << TIM_CR1_URS_Pos);    // Update event
    generated by any source
45  TIM3->CR1 &= ~(1 << TIM_CR1_DIR_Pos);    // Upcounting
    direction check
46  TIM3->CR1 &= ~(0x03 << TIM_CR1_CMS_Pos);    // (00) Edge
    aligned mode check
47  TIM3->CR1 |= (1 << TIM_CR1_ARPE_Pos);    // Auto reload
    preload enabled
48
49  TIM3->CCMR1 &= ~(0x03 << TIM_CCMR1_CC1S_Pos);    // TIM3_CH1
    configured as output
50  TIM3->CCMR1 |= (1 << TIM_CCMR1_OC1PE_Pos);    // Preload CCR1
    register aswell
51  TIM3->CCMR1 |= (0x6 << TIM_CCMR1_OC1M_Pos);    // PWM Mode 1
52
53  TIM3->CCER |= (1 << TIM_CCER_CC1E_Pos);    // Enable
    output channel 1
54  //TIM3->CCER |= (1 << TIM_CCER_CC1P_Pos);    // Active
    low
55
56
57
58  TIM3->PSC = 42 - 1;    // CNT incremented every ms
59  TIM3->ARR = 1000 - 1;    // Update event every
    seconds
60  TIM3->CCR1 = 500; //partiamo dalla ventola spenta e poi gli
    diamo potenza volta per volta
61
62
63
64  TIM3->EGR |= (1 << TIM_EGR_UG_Pos);    // Fire an update
    event to update shadow registers

```

Listing 1.2: implementation

1. TIM6 (Microsecond Timebase): Configured as a high-precision stopwatch for the ultrasonic sensors. The peripheral clock (APB1 at 84 MHz) is divided by a prescaler ($PSC = 83$), yielding exactly a 1 MHz counter frequency ($1\ \mu s$ resolution). It is utilized both to generate the precise 10 μs Trigger pulse (via the UIE update interrupt) and to measure the Echo signal's time of flight during the EXTI interrupt service routines.
2. TIM3 (PWM Generator): Dedicated to generating the control signal for the TB6612FNG motor driver. By setting the prescaler to 41 and the Auto-Reload Register (ARR) to 999, the timer produces a 2 kHz frequency wave. Configured in PWM Mode 1 (CCMR1 register), the duty cycle is dynamically updated in real-time by the PID controller writing directly to the CCR1 capture/compare register, mapping the 0-999 output to a 0-100% duty cycle.

Interrupts

To maintain a deterministic real-time control loop, asynchronous events are handled through the Nested Vectored Interrupt Controller (NVIC) and the External Interrupt/Event Controller (EXTI).

The first one handles all the interruptions of the system(internal and external), values the priority and handles the nesting between all the interrupts; in fact it's possible to observe that the NVIC is located near the core, in this way the latency is minimum.

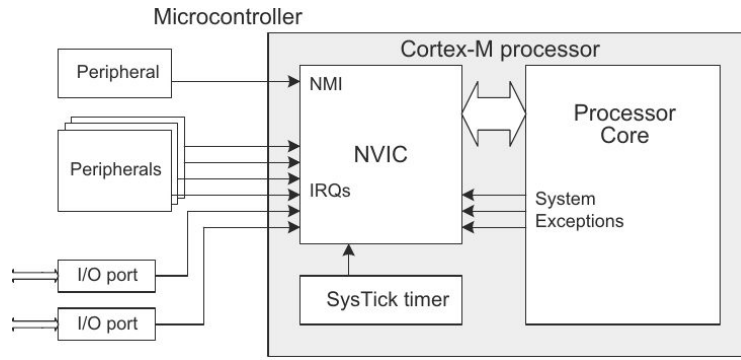


Figura 1.7: NVIC

Instead the EXTI is an independent hardware peripheral, that monitors changes in the electrical state of the microcontroller pins; since the pins are a lot, the EXTI groups them in line, when it detects the hardware event it raises a flag and forwards the request to the NVIC. In this project the ECHO pin of the sensor of the tube is connected at PC1 so we are using the EXT1 line, instead for the hand is used the EXT3 because is connected at PC3.

In particular :

1. Asynchronous Echo Interception: The Echo pins of the two HC-SR04 sensors (PC1 and PC3) are routed to the hardware lines EXTI1 and EXTI3 via the SYSCFG multiplexer. The trigger is enabled on both rising and falling edges (RTSR and FTSR registers). A rising edge starts TIM6, while a falling edge stops it, allowing asynchronous measurement of the acoustic flight time without halting the main while(1) loop.
2. Non-Blocking Serial Transmission: Telemetry data is exported via USART2 using the Transmit Data Register Empty (TXE) interrupt. Instead of utilizing blocking HAL functions, the system pushes one byte at a time into the Data Register (DR) only when the hardware is ready, fully decoupling the UART baud rate delays from the strict PID execution time.
3. Pending IRQ Clearing: Within every Interrupt Service Routine (ISR), the NVIC_ClearPendingIRQ() function and specific peripheral status flags (such as EXTI->PR for external interrupts or TIM6->SR for ti-

mer updates) are explicitly cleared to prevent the CPU from entering infinite interrupt loops.

```
1
2 // --- INTERRUPT SENSORI ---
3 void EXTI1_IRQHandler() { // TUBO
4     NVIC_ClearPendingIRQ(EXTI1_IRQn);
5     if((GPIOC->IDR >> 1) & 0x01) { //Legge il registro dei dati
6         in ingresso (IDR) della porta C, sposta a destra il valore
7         per isolare il bit 1 (PC1) e
8         //applica una maschera AND. Questo serve al
9         microcontrollore per capire se l'interrupt stato generato
10        da un fronte di salita (pin HIGH) o di discesa (pin LOW).
11        //se l'impulso echo andato alto allora :
12        TIM6->ARR = 65000; TIM6->CNT = 0; TIM6->CR1 |= (1 <<
13        TIM_CR1_CEN_Pos); //accendo il timer
14        measure_collected_tubo = 0;
15    } else { //il ping acustico tornato indietro
16        TIM6->CR1 &= ~(1 << TIM_CR1_CEN_Pos);
17        measure_tubo = TIM6->CNT; measure_collected_tubo = 1; //
18        spengo il timer, salvo il tempo accumulato e measure
19        collected va a 1
20    }
21    EXTI->PR |= (0x01 << EXTI_PR_PR1_Pos); //azzero il flag di
22    interruzione pendente
23 }
24
25 //questa funzione si comporta esattamente come la precedente
26 con la differenza che in questo caso stiamo gestendo il
27 canale 3 e contiamo la distanza della mano
28 void EXTI3_IRQHandler() { // MANO
29     NVIC_ClearPendingIRQ(EXTI3_IRQn);
30     if((GPIOC->IDR >> 3) & 0x01) {
31         TIM6->ARR = 65000; TIM6->CNT = 0; TIM6->CR1 |= (1 <<
32         TIM_CR1_CEN_Pos);
33         measure_collected_mano = 0;
34     } else {
35         TIM6->CR1 &= ~(1 << TIM_CR1_CEN_Pos);
36         measure_mano = TIM6->CNT; measure_collected_mano = 1;
37     }
38 }
```

```

27  EXTI->PR |= (0x01 << EXTI_PR_PR3_Pos);
28 }

```

Listing 1.3: implementation

1.3.2 Sensor Reading Implementation

Triggering Phase Unlike architectures that rely on hardware PWM to generate the trigger signal, this project utilizes direct GPIO manipulation synchronized by a microsecond-resolution timebase (TIM6). The customized measuring functions (e.g., `meas_dist_tubo()`) strictly enforce the timing requirements of the HC-SR04 module. The Trigger pin is initially driven LOW to ensure a clean electrical state, pulled HIGH for exactly $10\ \mu\text{s}$ to command the sonic burst, and finally pulled LOW again. This precise sequential toggling is governed by the `delay_micro_s()` function, which halts the specific execution thread just long enough to respect the sensor’s physical constraints, ensuring a deterministic start to the measurement cycle.

Measurement Phase The time-of-flight measurement relies entirely on the External Interrupt/Event Controller (EXTI) coupled with TIM6, rather than utilizing a timer in Input Capture mode. When the ultrasonic module emits the 40 kHz burst, its Echo pin transitions to HIGH. The EXTI hardware immediately detects this rising edge and fires the corresponding Interrupt Service Routine (ISR). Within the ISR, the microcontroller resets TIM6 to zero and enables its counter. While the acoustic wave travels through the air, the CPU is free to execute other tasks. Once the wave reflects off the obstacle and returns, the Echo pin drops to LOW. The EXTI detects this falling edge and triggers the ISR a second time. The software immediately halts TIM6 and saves the accumulated counter value. This final value represents the exact time of flight in microseconds, which is subsequently converted into a physical distance using the speed of sound.

1.3.3 Driver tb6612fng

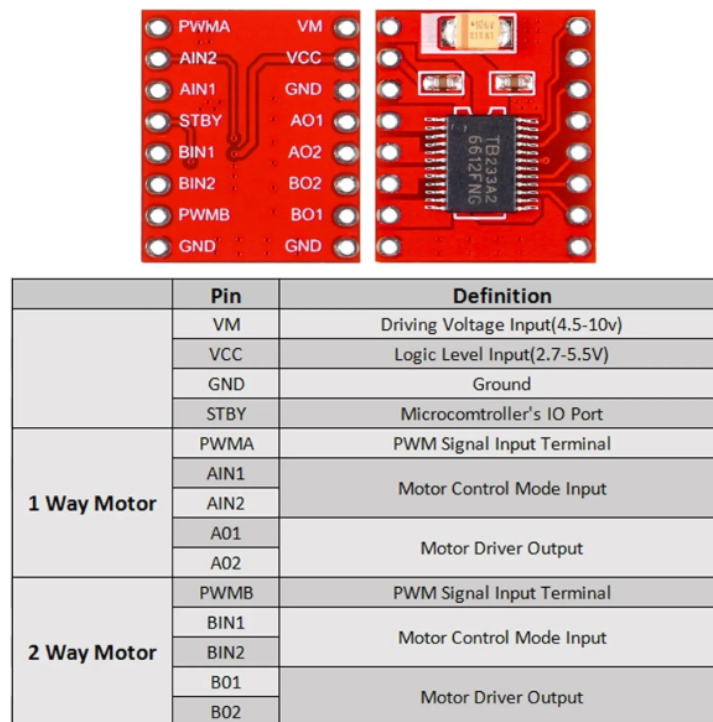


Figura 1.8: Driver

The TB6612FNG dual motor driver acts as the power interface between the low-current microcontroller logic and the high-current 5015 centrifugal fan. Utilizing a MOSFET-based H-bridge architecture, it effectively amplifies the PWM signal generated by TIM3 without the significant voltage drops and heat dissipation typical of older bipolar drivers (such as the L298N). The STM32 governs the actuation via the standby pin (PA4) and the directional logic pins (AIN1, AIN2).

1.3.4 Buck converter lm2596

Pinout:

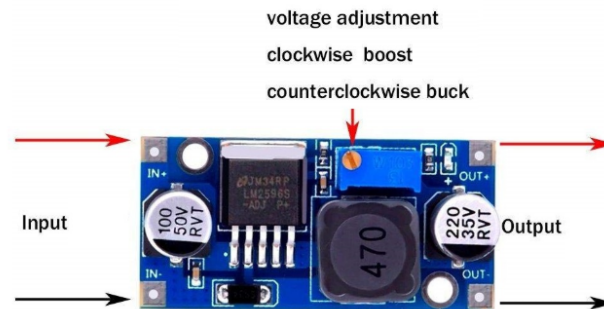


Figura 1.9: Buck Converter

The system is powered by an 8-cell AA battery pack providing approximately 12V. To safely supply the 5V required by the STM32 logic and the HC-SR04 sensors, an LM2596 DC-DC buck converter is employed. This step-down switching regulator is highly efficient and physically isolates the logic power supply from the motor's direct 12V rail. This decoupling is critical to prevent voltage spikes and electromagnetic noise generated by the fan from resetting or freezing the microcontroller during operation.

Capitolo 2

Serial Communication and Filter

2.1 Serial Communication (USART)

The system utilizes the Universal Asynchronous Receiver and Transmitter (USART) paradigm to transmit real-time telemetry data to a PC terminal. This asynchronous communication relies on a continuous transmission of bits without a synchronizing clock, requiring a strict agreement on transmission parameters between the sender and receiver.

The USART2 peripheral is initialized with an 8-bit word length, no parity bit, and 1 stop bit. To prevent the serial communication from bottlenecking the real-time control loop, the baud rate is set to 115200 bps. This is calculated based on the peripheral bus clock frequency (f_{ck}) and utilizes a hardware oversampling by 16 to ensure optimal signal tolerance.

$$baud_rate = \frac{f_{ck}}{8 \cdot (2 - Over8) \cdot UsartDIV}$$

Where a hardware oversampling by 16 ($Over8 = 0$) is applied to ensure optimal signal tolerance against noise[.

To maintain the deterministic execution time of the aerodynamic PID controller, standard blocking HAL transmission functions (e.g., `HAL_UART_Transmit`) are completely bypassed. Instead, the system pushes one byte at a time into the Data Register (DR) only when the hardware is ready, fully decoupling the

UART baud rate delays from the strict PID execution time. The custom transmission logic is handled autonomously by the TXE (Transmit Data Register Empty) interrupt:

```

1 void USART2_IRQHandler() {
2     NVIC_ClearPendingIRQ(USART2_IRQn);
3     // Check if the Transmit Data Register is empty
4     if(tx_len < tx_len_max && (USART2->SR >> USART_SR_TXE_Pos)
    & 0x01) {
5         USART2->DR = buff_tx[++tx_len]; // Send next byte
6     } else if((USART2->SR >> USART_SR_TC_Pos) & 0x01) {
7         tx_len = 0;
8         // Transmission complete, disable TXE interrupt
9         USART2->CR1 &= ~(0x01 << USART_CR1_TXEIE_Pos);
10    }
11 }

```

Listing 2.1: Non-blocking USART Transmission via Interrupt

The exported telemetry string is formatted as a comma-separated values (CSV) array containing the filtered hand distance, the filtered ball distance, and the current PWM output. This format allows immediate real-time plotting and post-processing in MATLAB.

Here it's possible to observe how works the interface with **realterm** :

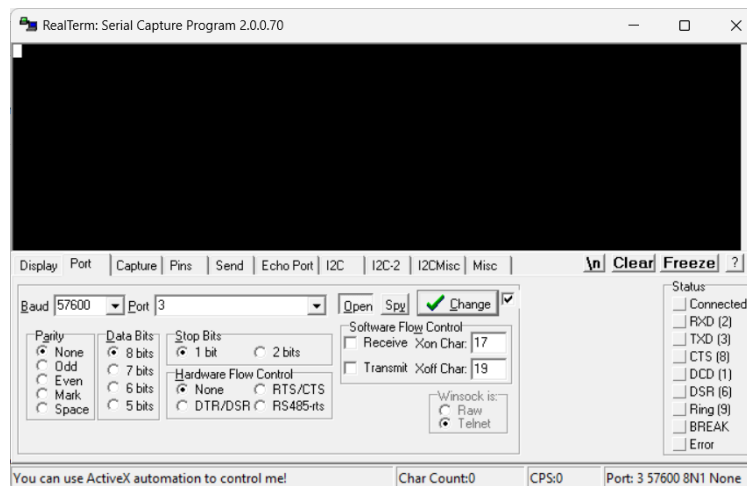


Figura 2.1: RealTerm port

This is the first window, we need to go to **port** in order to visualize the data in the way that is writtern on the code, then insert the right baud rate used and the port related to the f446re.

2.2 Data Filtering (EMA)

Most of the time, due to noisy sensors or acoustic reflections inside the tube, the measurements accuracy needs to be improved. To mitigate high-frequency acoustic noise and occasional spurious readings, the raw distance measurements are smoothed using an Exponential Moving Average (EMA) filter.

Unlike standard complementary filters, this infinite impulse response (IIR) discrete filter computes the new stable distance based strictly on the current raw sample and the previous filtered value. The mathematical implementation follows the difference equation:

$$y_k = \alpha \cdot x_k + (1 - \alpha) \cdot y_{k-1}$$

Where y_k is the new filtered distance, x_k is the raw sensor reading, and y_{k-1} is the previously filtered state. In this firmware, the smoothing factor α is set to 0.2. This weight heavily favors the historical data over new incoming samples, significantly dampening mechanical vibrations at the cost of a minimal, physiologically acceptable phase delay during rapid hand movements.

```
1 // EMA Filter applied to the hand sensor reading
2 if (dist_filtrata_mano == 0.0f) {
3     dist_filtrata_mano = raw_mano; // Initialization
4 } else {
5     dist_filtrata_mano = (alpha * raw_mano) + ((1.0f - alpha) *
6         dist_filtrata_mano);
7 }
```

Listing 2.2: EMA Filter Implementation

Capitolo 3

PID controller

3.1 Control Strategy and PID Implementation

The core aerodynamic levitation is managed by a hybrid control architecture combining a Finite State Machine (FSM) for safety and a discrete Proportional-Integral-Derivative (PID) controller for active tracking.

Finite State Machine (FSM)

The execution flow evaluates the physical state of the system at each iteration to enforce hardware safety:

- **Idle State:** If the lower sensor fails to detect the user's hand for more than 5 consecutive control cycles, the system assumes the operator has left. The control effort is forced to zero and the PID integral term is reset to prevent windup upon restart.
- **Kickstart State:** When the ball rests entirely at the bottom of the tube (> 45.0 cm), a maximum thrust command ($PWM = 999$) is temporarily applied to overcome static friction.
- **Tracking State:** Engaged during standard free-flight. The target set-point is calculated via geometric inversion (50.0 cm minus the filtered hand distance), bounded by a hardcoded 5 cm safety ceiling.

Discrete PID Controller

Once in the Tracking State, the discrete PID algorithm computes the required fan duty cycle based on a fixed time step ($DT = 0.06$ s). To avoid the phenomenon of windup when the control input reaches the physical limits of the

system, an anti-windup strategy based on clamping was implemented. If the system saturates (reaching PWM_{MAX} or PWM_{MIN}), the integral accumulation is temporarily suspended, preventing overshoot and excessive settling time.

Furthermore, to prevent violent control spikes ("derivative kicks") caused by step changes in the user's hand position, the derivative action ignores the setpoint error entirely. Instead, it is computed solely on the velocity of the ball (Feedback Gradient). The complete C implementation is shown below:

```

1 float calcola_PID(float setpoint, float posizione) {
2     float errore = posizione - setpoint;
3     float P = Kp * errore;
4
5     // Anti-windup (Clamping Method)
6     float out_parziale = PWM_BASE + P + (Ki * integrale);
7     if ((out_parziale > PWM_MAX && out_parziale < PWM_MIN) ||
8         (out_parziale >= PWM_MAX && errore < 0) ||
9         (out_parziale <= PWM_MIN && errore > 0)) {
10         integrale += errore * DT;
11     }
12     float I = Ki * integrale;
13
14     // Derivative on Measurement (Anti-Kick)
15     float derivata_pos = (posizione - pos_pallina_prec) / DT;
16     float D = Kd * derivata_pos;
17
18     // Final Output Calculation
19     float out_finale = PWM_BASE + P + I + D;
20
21     // Output Saturation
22     if (out_finale > PWM_MAX) out_finale = PWM_MAX;
23     if (out_finale < PWM_MIN) out_finale = PWM_MIN;
24
25     pos_pallina_prec = posizione;
26     return out_finale;
27 }

```

Listing 3.1: Discrete PID with Clamping Anti-Windup and Anti-Kick

The values of K_p , K_d and K_i were chosen empirically, running various tests to best understand how everything works.

3.1.1 Proportional, derivative and integral gain

Proportional gain

K_p is the proportional gain and is responsible for managing reactivity, controlling the instantaneous thrust force. This is because it acts directly on the error, or rather, the distance between the ball and the target, multiplying it. If the ball, for example, is far below the target, this will generate a much greater force, a very strong upward thrust. As the ball gets closer to the target, the force decreases, and therefore the proportional thrust decreases. However, this alone is not enough, as there will always be some steady-state error. For my project the best value of k_p is 13.

Derivative gain

K_d is the derivative action; it is used to provide stability and prevent the ball from rapidly jumping up and down around the set point during the settling period. It can actually be used as an aerodynamic dampener. However, care must be taken when using this value, because if there is too much background noise (perhaps from sensors or some external disturbance) and this parameter is incorrect, the opposite effect will be achieved, resulting in a so-called derivative kick. In the project : K_d is 6.0.

Integral action

K_i is the precision, the memory of the system; it accumulates the error over time based on the sampling time DT . If the ball stagnates a couple of centimeters below the target, the integral notices this, adds this small continuous error and gently raises the base PWM until the ball touches the exact millimeter.

Capitolo 4

Graphics evaluation

In this chapter, the real-time telemetry data exported via the non-blocking USART interface is analyzed to evaluate the dynamic performance of the control loop. Figure displays the system response (top) and the corresponding control effort (bottom) over a continuous operational window.

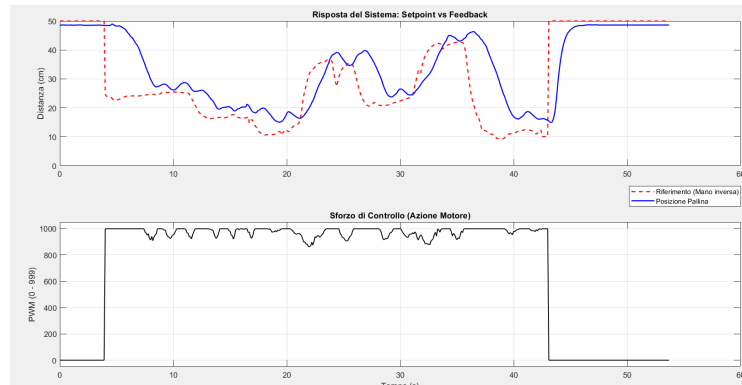


Figura 4.1: Graph

Setpoint Tracking and System Response

The upper plot illustrates the dynamic setpoint (dashed line), dictated by the user's hand position, and the actual position of the levitating ball (solid line). As observed, the system successfully tracks continuous and unpredictable variations in the reference height. When the setpoint undergoes a rapid change, the ball reaches the new target with an acceptable settling time. A minor phase delay and occasional slight overshoots are visible; these are expected physical behaviors caused by the inherent turbulence of the airflow inside the tube and the intentional smoothing effect of the EMA filter ($\alpha = 0.2$) applied to the raw acoustic data.

Control Effort and Actuator Dynamics

The lower plot represents the PWM duty cycle applied to the centrifugal blower fan (ranging from 0 to 1000). The control effort is highly reactive: sharp increases in the setpoint correspond to immediate spikes in the PWM output, driven primarily by the proportional (K_p) action. Conversely, when the target height drops rapidly, the derivative-on-measurement (K_d) action and the clamping anti-windup logic successfully prevent instability, often dropping the PWM to its minimum saturation limit to allow gravity to pull the ball down naturally.

Furthermore, the integration of the Finite State Machine (FSM) is clearly visible in the actuator's behavior. For instance, the absolute drops to zero PWM indicate the activation of the *Idle State* (when the hand is removed from the sensor's field of view), while saturation peaks at maximum PWM represent the *Kickstart State* engaging to overcome the ball's initial static friction.

Capitolo 5

Conclusion and Future Developments

5.1 Project Conclusions

This project successfully demonstrated the design and implementation of an active aerodynamic levitation system, driven entirely by a bare-metal firmware architecture on the STM32 Nucleo-F446RE microcontroller.

The custom integration of EXTI-driven ultrasonic measurements and non-blocking USART telemetry ensured that the CPU remained free to execute time-critical mathematical operations. Furthermore, the application of an Exponential Moving Average (EMA) filter significantly stabilized the acoustic readings, providing clean feedback for the control algorithm. The core objective—maintaining a levitating object at a dynamic, user-dictated target height—was accomplished through a hybrid Finite State Machine and a discrete PID controller. The addition of clamping anti-windup logic and derivative-on-measurement (anti-kick) techniques effectively compensated for the inherent non-linearities of fluid dynamics and sudden setpoint variations, resulting in a responsive and stable levitation.

5.2 Future Developments and Industrial Applications

While the current architecture fully satisfies the initial project requirements, the system’s modularity opens several avenues for future technical enhancements and scalability into real-world applications.

Hardware and Software Upgrades

- **Sensor Upgrades:** Replacing the ultrasonic modules with Time-of-Flight (ToF) optical sensors (such as the VL53L0X) would entirely eliminate acoustic interference and refraction issues, enabling sub-millimeter precision at higher sampling rates.
- **Advanced Control Strategies:** The current PID architecture could be upgraded by modeling the physical plant mathematically to implement modern control strategies, such as a Linear-Quadratic Regulator (LQR) or Model Predictive Control (MPC), to better predict and manage aerodynamic turbulence.

Industrial Applications

The fundamental principles validated in this embedded prototype are directly translatable to several advanced industrial fields:

- **Contactless Material Handling:** The same aerodynamic control loop can be scaled to handle highly delicate or sterile materials—such as semiconductor wafers, pharmaceutical capsules, or chemical compounds—preventing physical contamination or mechanical stress.
- **Pneumatic Transport Systems:** The algorithms developed for real-time airflow modulation are applicable to industrial pneumatic tubes, optimizing the velocity and positioning of transport capsules across complex networks.
- **Active Suspension Systems:** The rapid feedback-and-actuation paradigm serves as a foundation for active air suspension systems in the automotive and aerospace sectors, where instantaneous compensation for external physical disturbances is critical.

Bibliografia

- [1] TB6612FNG_datasheet_en_20141001
- [2] lm2596_datasheet
- [3] stm32Nucleof446_reference_manual